OpenAI Agents SDK

The simplest correct model is this: the SDK is a control layer for building agentic systems where the model can call tools, transfer control to other agents, use sessions for continuity, and produce traceable runs. OpenAI's docs explicitly say `Agent` plus `Runner` lets the SDK manage turns, tools, guardrails, handoffs, and sessions; if you want to own that loop yourself, you use the Responses API directly. (OpenAI GitHub)

What follows is the practical version: what each concept means, what value it gives, where to use it, and where people break things.

---

1) Agent

An **Agent** is not "the whole app." It is one configured LLM role: instructions, tools, guardrails, handoffs, and model settings. The docs describe an agent as an AI model configured with instructions, tools, guardrails, handoffs, and more, and strongly recommend passing `instructions` because they become the system prompt for that agent. (OpenAI GitHub)

What it means in real life

Think of an agent like a trained employee with a job description.

A hospital intake agent might be told:

- ask for symptoms,
- collect duration and severity,
- detect red flags,
- hand off billing questions,
- never invent diagnoses.

That is an agent. It is not the whole hospital; it is one specialist role.

Value of the concept

Without agents, you usually end up with one giant prompt that tries to do everything. That becomes brittle fast. With agents, you split responsibilities cleanly:

- one agent handles triage,
- one handles coding,
- one handles billing,
- one handles scheduling.

That separation improves control, testing, and reuse. The docs also support mixing providers per agent, so one specialist can use one model and another specialist can use another model if needed. (OpenAI GitHub)

When to use it

Use an agent when you need a distinct role:

- support bot
- research bot
- coding assistant
- medical intake assistant
- exam tutor
- sales qualifier

Edge case

Do not make an “agent” for every tiny subtask. That is over-fragmentation. If the task is simple, a single agent plus a few tools is cleaner than a maze of handoffs.

---

2) Runner

The **Runner** is the execution loop. It runs the agent, handles tool calls, handles handoffs, and keeps the run going until the system reaches a final answer or an interruption. The docs say the runner handles executing individual agents, any handoffs, and any tool calls, and that a single run can contain multiple LLM calls while still representing one logical user turn. (OpenAI GitHub)

What it means in real life

The runner is the dispatcher, not the worker.

Imagine a clinic front desk system:

1. user asks a question,
2. the triage agent reads it,
3. the agent calls a symptom-check tool,
4. the agent hands off to billing if needed,
5. the final answer comes back.

The runner is what keeps that whole chain coordinated.

Value of the concept

The runner saves you from hand-writing the boring, error-prone loop:

- call model
- parse tool call
- execute tool
- feed result back
- maybe hand off
- repeat

That loop is where most agent apps become messy.

When to use it

Use `Runner.run(...)` for normal agentic applications. Use the lower-level Responses API directly only when you want to own every step yourself. (OpenAI GitHub)

Edge case

A single `Runner.run(...)` is not a single model call. It can be several model calls inside one user turn. If you log or budget poorly, you will underestimate latency and token usage. (OpenAI GitHub)

---

3) Tools

Tools are how the agent acts. The docs say tools let agents fetch data, run code, call external APIs, and even use a computer. The SDK supports five categories: hosted tools, local/runtime tools, Python function tools, agents-as-tools, and the experimental Codex tool. (OpenAI GitHub)

Real-life meaning

A model can talk. A tool lets it do something:

- check inventory
- search policy docs
- call a billing API
- calculate dosage boundaries
- query a database
- open a file
- generate a spreadsheet
- search the web

Value of the concept

This is the point where AI stops being a text generator and becomes a system component. Tools are the bridge from “language” to “action.”

When to use it

Use a tool whenever the agent needs:

- live data
- deterministic computation
- external side effects
- authority checks
- file access
- domain-specific logic

Edge case

Do not give an agent unrestricted tools just because you can. That turns a controlled system into a liability. Restrict tools tightly and make each one do one thing well.

---

4) Function tools

A **function tool** is a normal Python function the SDK exposes to the model. The SDK builds the tool schema from the function signature and docstring. The docs state that type annotations and docstrings are used to

generate the schema, and `Field` constraints can refine arguments. (OpenAI GitHub)

Real-life example

A pharmacy assistant agent might have:

```
@function_tool
def check_stock(medicine_name: str) -> str:
    """Check whether a medicine is in stock."""
    ...
```

The model does not “know medicine stock.” It uses the function tool to ask your inventory service.

Value of the concept

Function tools are the best way to connect your own code to the model:

- your database logic
- your business rules
- your internal APIs
- your calculations

They are also easy to test because they are just Python functions.

When to use it

Use function tools for:

- database queries
- calculator logic
- policy checks
- formatting
- API calls
- local business operations

Edge case

If the function returns huge raw text or giant JSON, you will bloat the conversation and waste tokens. Trim or summarize tool output before feeding it back where possible. The SDK includes a `ToolOutputTrimmer` extension specifically for this kind of problem. (OpenAI GitHub)

---

5) Hosted tools

Hosted tools are OpenAI-managed tools that run on the platform side. The tools docs list hosted tools such as web search, file search, code interpreter, hosted MCP, and image generation. (OpenAI GitHub)

Real-life meaning

This is useful when you want the SDK to use built-in capabilities instead of wiring everything yourself.

Examples:

- a support assistant that searches your knowledge base,
- a research workflow that retrieves documents,
- a data analysis assistant that uses code execution.

Value of the concept

Hosted tools reduce integration work and standardize behavior. They are useful when speed to prototype matters.

Edge case

Hosted tools are not a substitute for your own application logic. If the system needs a strict business rule, do not hide it in a generic retrieval step.

---

6) Handoffs

A **handoff** is when one agent delegates to another agent. OpenAI's docs say handoffs are useful when different agents specialize in distinct areas, and that they are represented as tools to the LLM. A handoff to `Refund Agent` appears to the model as a tool like `transfer_to_refund_agent`. (OpenAI GitHub)

Real-life meaning

This is a specialist transfer.

Examples:

- triage agent → billing agent
- general support → technical support
- admissions bot → scholarship bot
- medical intake → medication safety reviewer

Value of the concept

Handoffs let you build specialist systems without making one agent do everything. That improves:

- accuracy
- maintainability
- separation of concerns
- auditability

When to use it

Use handoffs when the next agent should actually take over the conversation, not merely assist in the background. (OpenAI GitHub)

Edge case

Do not hand off too aggressively. If every minor question triggers a transfer, the user experience becomes fragmented. Keep a clear triage policy.

---

7) Agents-as-tools

This is the pattern where one agent calls another agent like a tool, but stays in control of the overall orchestration. OpenAI explicitly distinguishes this from handoffs in the agent orchestration docs. (OpenAI GitHub)

Real-life meaning

A manager agent stays on top. Specialists are consulted, but the manager decides the final answer.

Example:

- manager agent asks a legal specialist for a clause interpretation,
- asks a finance specialist for a number,
- then composes the final response itself.

Value of the concept

This is useful when you want:

- one consistent voice,
- one final policy owner,
- stronger control over the response format.

When to use it

Use it when the outer agent must preserve responsibility and merely delegate subwork.

Edge case

This can create hidden complexity if the manager agent becomes too dependent on specialists. In that case, make the specialist output strictly structured.

---

8) Guardrails

Guardrails are checks on input and output. The docs define two kinds: input guardrails run on the initial user input, and output guardrails run on the final agent output. The docs also give a concrete motivation: use a fast, cheap guardrail model to block malicious or irrelevant requests before a slower, expensive model does work. (OpenAI GitHub)

Real-life meaning

Guardrails are your safety and policy filters.

Examples:

- reject requests for disallowed content,

- block medical advice outside scope,
- stop prompt injection attempts,
- detect nonsense inputs,
- validate that the final answer matches a required schema.

Value of the concept

Guardrails save money, reduce risk, and enforce policy before damage is done.

When to use it

Use guardrails whenever:

- user content is sensitive,
- tool access is powerful,
- output must follow a regulated format,
- bad inputs are likely.

Edge case

Guardrails are not a perfect security boundary. They help, but they are not a replacement for tool-level authorization and backend validation.

---

9) Sessions

Sessions provide memory across runs. The docs say the SDK provides built-in session memory to maintain conversation history across multiple agent runs, so you do not have to manually pass `.to_input_list()` every turn. (OpenAI GitHub)

Real-life meaning

A session is the conversation container.

Example:

- the user says, “I have chest pain”
- later says, “it started yesterday”
- later says, “what does that mean?”

The session remembers the earlier context.

Value of the concept

Without sessions, multi-turn UX becomes painful. You would have to manually preserve and reconstruct history every time.

When to use it

Use sessions for:

- chat apps

- tutoring apps
- intake flows
- support conversations
- any multi-step workflow

Edge case

Sessions are not the same thing as server-side continuation modes. The docs explicitly caution against mixing session memory with `conversation_id`, `previous_response_id`, or `auto_previous_response_id` in the same run. (OpenAI GitHub)

That means: pick one memory strategy per application path. Do not stack three memory systems on top of each other and expect sanity.

---

10) Results

The **result object** is what you get back after a run. The docs say the most useful surfaces are:

- `final_output`
- `to_input_list()`
- `new_items`
- `last_agent`
- `last_response_id`
- `interruptions`
- `to_state()` (OpenAI GitHub)

Real-life meaning

This is not just “the answer text.” It is the evidence trail and continuation state.

Value of the concept

You need results because a production system often cares about more than the final answer:

- what tools were called,
- which agent handled the turn,
- whether the run paused for approval,
- what state should be resumed later.

When to use it

Use different result surfaces for different jobs:

- `final_output` for the user
- `new_items` for logs or UI playback
- `last_agent` for routing the next turn
- `to_input_list()` when you manage conversation history yourself (OpenAI GitHub)

Edge case

Do not assume `final_output` is always a string. The docs say it can be an object if the agent has an `output_type`, and it can be `None` if the run stopped before final output due to an interruption. (OpenAI GitHub)

---

11) Tracing

Tracing records what happened during the run. The docs say the SDK includes built-in tracing for LLM generations, tool calls, handoffs, guardrails, and custom events, and that tracing is enabled by default. (OpenAI GitHub)

Real-life meaning

Tracing is your flight recorder.

If the agent gives a wrong answer, tracing tells you:

- which agent decided,
- which tool was called,
- what the tool returned,
- where the chain broke.

Value of the concept

This is the difference between “something went wrong” and “I know exactly why it went wrong.”

When to use it

Always use tracing in development and production unless you have a clear reason not to. The docs say it can be disabled, including in environments with Zero Data Retention policy. (OpenAI GitHub)

Edge case

If you build agent systems without tracing, you are debugging blind. That is a mistake.

---

12) Model selection

The SDK lets you choose models at different levels. The docs say `Agent.model` can specify a model on a specific agent, and `Runner.run` can use a model provider at the run level, which allows different providers across agents. (OpenAI GitHub)

Real-life meaning

Not every agent deserves the same model.

Example:

- triage agent uses a cheap fast model,
- specialist clinical reasoning agent uses a stronger model,
- summarizer uses a small model.

Value of the concept

This is where cost and latency control come from. You do not pay premium compute for every trivial step.

When to use it

Use different models when tasks differ in:

- complexity
- cost sensitivity
- latency sensitivity
- reliability requirements

Edge case

Overusing a large model for routing is wasteful. Underusing a weak model for high-stakes reasoning is dangerous. Match model strength to task strength.

---

13) Sandbox agents

Sandbox agents are for work that needs an isolated workspace with files, shell access, snapshots, and sandbox-native capabilities. The docs describe them as an execution harness that avoids having you wire together file staging, filesystem tools, shell access, sandbox lifecycle, snapshots, and provider glue yourself. (OpenAI GitHub)

Real-life meaning

This is for tasks like:

- editing a repo,
- analyzing a codebase,
- processing files,
- running commands safely,
- generating artifacts in a controlled environment.

Value of the concept

A sandbox gives the agent a real workspace instead of pretending text is enough.

When to use it

Use sandbox agents when the task requires:

- files,
- code execution,
- staged artifacts,
- reproducible filesystem state.

Edge case

Do not try to simulate a workspace by stuffing file contents into chat if the task is file-heavy. That is inefficient and fragile.

---

14) Realtime and voice agents

The SDK also supports realtime and voice workflows. The realtime quickstart says realtime agents in the Python SDK are server-side, low-latency agents built on the Realtime API over WebSocket transport, and the voice pipeline is a three-step flow: speech-to-text, your agentic workflow, then text-to-speech. The docs also note the realtime feature is beta. (OpenAI GitHub)

Real-life meaning

This is for:

- call centers
- voice assistants
- telephony
- live tutoring
- spoken medical intake

Value of the concept

You get conversational latency and voice UX without writing every transport layer yourself.

Edge case

Realtime is not the default path. It is a specialized path for voice and low-latency interactions, not a replacement for normal text-based agent workflows. (OpenAI GitHub)

---

15) How all the pieces work together

A realistic system usually looks like this:

1. **Input arrives** User asks something.
2. **Input guardrail checks it** Blocks unsafe or irrelevant content. (OpenAI GitHub)
3. **Triage agent reads the request** It decides whether to answer directly or hand off. (OpenAI GitHub)
4. **Tools are called if needed** Inventory, search, database, calculator, API, file lookup. (OpenAI GitHub)
5. **Runner keeps the loop moving** It handles tool calls and agent transfers until completion. (OpenAI GitHub)
6. **Output guardrail checks the final response** Prevents bad output from reaching the user. (OpenAI GitHub)
7. **Result object records the run** You inspect final output, last agent, new items, or state. (OpenAI GitHub)

8. **Tracing captures everything** You debug the run from the dashboard. (OpenAI GitHub)

That is the framework.

---

16) Concrete real-life use cases

Customer support

- **Agent**: general support
- **Handoff**: billing, refunds, technical support
- **Tools**: order lookup, ticket creation, FAQ search
- **Guardrails**: block abuse, enforce policy
- **Sessions**: remember the customer thread
- **Tracing**: audit complaints and tool failures (OpenAI GitHub)

Healthcare intake

- **Agent**: symptom intake
- **Handoff**: pharmacy, billing, clinical escalation
- **Tools**: appointment lookup, red-flag checker, policy retrieval
- **Guardrails**: prevent unsafe advice
- **Sessions**: remember the conversation
- **Tracing**: audit decision path (OpenAI GitHub)

Medical education tutor

- **Agent**: tutor
- **Tools**: case search, flashcard generator, rubric scorer
- **Guardrails**: stop hallucinated certainty in high-stakes content
- **Sessions**: track learner progress
- **Results**: structured feedback and next-step plan (OpenAI GitHub)

Coding assistant

- **Agent**: code reviewer
- **Sandbox agent**: run code in isolated workspace
- **Tools**: file editing, tests, linting, repository inspection
- **Tracing**: see why a patch was produced
- **Results**: final patch plus artifacts (OpenAI GitHub)

Voice assistant

- **Realtime agent**: live conversation
- **Tools**: scheduling, search, action execution
- **Voice pipeline**: speech-to-text → agent workflow → text-to-speech (OpenAI GitHub)

---

17) The real value of the SDK

The value is not “AI can talk.” The value is:

- **structure**: you can define roles and responsibilities,
- **control**: you can restrict tool access and handoffs,
- **reuse**: you can build specialist agents once and reuse them,
- **observability**: tracing shows what actually happened,
- **continuity**: sessions preserve context,
- **cost control**: use smaller models and guardrails where appropriate,
- **extensibility**: tools and sandboxes connect the model to real systems. (OpenAI GitHub)

That is why the SDK matters. It turns an LLM from a text endpoint into a controlled orchestration layer.

---

18) Common mistakes

Mistake 1: one giant agent for everything

Bad design. You get tangled prompts and weak control. The docs explicitly support agent orchestration and handoffs because separation is the point. (OpenAI GitHub)

Mistake 2: giving too many tools

Bad design. You create unpredictable behavior and security risk. Use narrow tools.

Mistake 3: mixing memory systems

Bad design. Do not combine sessions with incompatible continuation patterns in the same run. (OpenAI GitHub)

Mistake 4: not using tracing

Bad design. You lose visibility into tool calls, handoffs, and guardrails. (OpenAI GitHub)

Mistake 5: using a heavy model for trivial routing

Bad design. Use model selection strategically. (OpenAI GitHub)

Mistake 6: dumping raw tool output back into context

Bad design. You bloat tokens and degrade performance. Use trimming or summarization. (OpenAI GitHub)

---

19) A practical rulebook

Use this rulebook when building with the SDK:

- One agent per clear role. (OpenAI GitHub)
- One tool per clear action. (OpenAI GitHub)
- One handoff only when the specialist should truly take over. (OpenAI GitHub)
- Guardrails for anything risky, regulated, or expensive. (OpenAI GitHub)
- Sessions for multi-turn memory. (OpenAI GitHub)
- Tracing always on during serious development. (OpenAI GitHub)
- Sandbox agents when the work needs files or commands. (OpenAI GitHub)
- Use smaller models for cheap routing, larger models for hard reasoning. (OpenAI GitHub)

If you want the next step, the useful move is not more theory. It is a **full architecture breakdown** of one real system built with these concepts: agents, tools, handoffs, guardrails, memory, tracing, and sandboxing mapped onto a healthcare or education workflow.

General Agents SDK architecture

The right way to think about the OpenAI Agents SDK is as a **workflow runtime** for LLM applications, not as a single chatbot API. OpenAI describes agents as applications that plan, call tools, collaborate across specialists, and keep enough state to complete multi-step work. The core runtime pattern is `Agent` plus `Runner`, which manages turns, tools, guardrails, handoffs, and sessions. (OpenAI Developers)

1) The architecture in one picture

```
User / API request
        ↓
Input guardrail
        ↓
Triage agent
   ├─ direct answer
   ├─ tool call
   ├─ handoff to specialist agent
   ├─ agent-as-tool call
   └─ pause for human approval
        ↓
Runner keeps executing the loop
        ↓
Output guardrail
        ↓
Result object + trace + session state
```

That is the real model. One user turn can contain multiple LLM calls, tool calls, and handoffs, but it is still one logical run. OpenAI explicitly says the runner can execute individual agents, handoffs, and tool calls inside a single turn, and multi-step runs can produce more than one response. (OpenAI GitHub)

---

2) Each layer and why it exists

A. Agent: the role

An agent is one configured model role with instructions, tools, handoffs, and guardrails. OpenAI's docs describe an agent as an AI model configured with instructions, tools, guardrails, handoffs, and more, and they recommend passing instructions because they become the system prompt for that agent. (OpenAI GitHub)

Why it matters: This gives you separation of responsibilities. Instead of one giant prompt doing everything, you create focused specialists.

Real-life value:

- support agent
- triage agent
- billing agent
- summarizer
- safety checker

B. Runner: the execution loop

The runner is the engine that keeps the system moving. It calls the model, handles tool execution, follows handoffs, and stops when a final answer is ready. (OpenAI GitHub)

Why it matters: You do not manually write the "call model → parse tool call → execute tool → feed result back → repeat" loop.

Real-life value: Less boilerplate, fewer broken orchestration paths, cleaner state handling.

C. Tools: the hands

Tools let agents take actions: fetch data, run code, call APIs, or use a computer. The SDK supports hosted tools, local/runtime tools, Python function tools, agents-as-tools, and the Codex tool. (OpenAI GitHub)

Why it matters: This is where the model stops being "just text" and becomes operational.

Real-life value:

- query a database
- check inventory
- fetch a patient record
- call a scheduling API
- validate a policy
- compute a result

D. Handoffs: delegation to specialists

Handoffs let one agent delegate to another. OpenAI says they are useful when different agents specialize in distinct areas, and they are represented as tools to the LLM. (OpenAI GitHub)

Why it matters: It keeps the system modular.

Real-life value: A general router can transfer a billing question to billing, a technical question to technical support, or a summary task to a summarizer.

E. Agents-as-tools: manager style orchestration

OpenAI also supports the pattern where the main agent stays responsible and calls specialist agents as helpers. This is the manager-style approach. (OpenAI Developers)

Why it matters: Use this when the main agent should retain final control but needs subwork done by specialists.

Real-life value: A manager agent gathers evidence from multiple subagents and then writes one final answer.

F. Guardrails: policy and safety checks

Guardrails validate user input and agent output. OpenAI explicitly says you can use a fast, cheap guardrail model to block malicious or irrelevant requests before spending on a slower, more expensive model. (OpenAI GitHub)

Why it matters: They reduce risk and wasted computation.

Real-life value:

- reject unsafe requests
- block prompt injection
- enforce format rules
- stop bad tool usage

G. Sessions: memory across turns

Sessions provide built-in conversation memory across runs, so you do not have to manually reconstruct history every turn. OpenAI says sessions maintain conversation history across multiple agent runs and eliminate manual `.to_input_list()` handling between turns. (OpenAI GitHub)

Why it matters: Without sessions, every turn becomes stateless unless you rebuild history yourself.

Real-life value: Chat apps, tutoring flows, workflows that pause and resume, multi-step forms.

H. Local context: application state that should not go to the model

OpenAI says the SDK lets you pass application state and dependencies into a run without sending them to the model, including authenticated user info, database clients, loggers, and helper functions. (OpenAI Developers)

Why it matters: You keep private or technical data out of model context while still making it available to tools.

Real-life value:

- user permissions

- DB handles
- request IDs
- logger
- feature flags

I. Results: what you get back

The result object contains more than the final text. OpenAI documents surfaces like `final_output`, `last_agent`, `last_response_id`, `raw_responses`, interruptions, and state helpers. (OpenAI GitHub)

Why it matters: You need the answer, but you also need the audit trail and continuation data.

Real-life value:

- render the final user response
- inspect which agent answered
- resume paused runs
- debug the chain of events

J. Tracing: observability

Tracing records LLM generations, tool calls, handoffs, guardrails, and custom events. It is enabled by default and is meant for debugging, visualization, and monitoring in development and production. (OpenAI GitHub)

Why it matters: Without tracing, you cannot reliably debug agent behavior.

Real-life value: You can see why the system chose a tool, why it handed off, or where it failed.

K. Human-in-the-loop: approvals

The SDK supports pausing execution until a person approves or rejects sensitive tool calls. The docs say sensitive tools can surface pending approvals as interruptions and resume later from `RunState`. (OpenAI GitHub)

Why it matters: This is essential for risky actions.

Real-life value: Payments, deletions, outbound messages, medical escalation, account changes.

L. Sandbox agents: isolated execution workspace

Sandboxes are for cases where the answer depends on work done in a workspace with files, commands, packages, ports, snapshots, and memory. OpenAI says to use a sandbox when the answer depends on sandbox work, not just prompt reasoning. (OpenAI Developers)

Why it matters: You separate orchestration from execution.

Real-life value:

- code edits

- file processing
- artifact generation
- dataset inspection
- reproducible workspace tasks

---

3) The standard control flow

This is the lifecycle of one run:

1. User sends input.
2. Input guardrail checks the request.
3. The first agent reads the request.
4. The agent either:
    - answers directly,
    - calls a tool,
    - hands off to another agent,
    - invokes a specialist as a tool,
    - or pauses for human approval.
5. The runner keeps looping until the run finishes.
6. Output guardrail checks the final answer.
7. Result object and trace are produced. (OpenAI GitHub)

That loop is the whole framework.

---

4) Core code pattern

Minimal single-agent pattern

```
import asyncio
from agents import Agent, Runner

agent = Agent(
    name="Assistant",
    instructions="Answer clearly and briefly."
)

async def main():
    result = await Runner.run(agent, "Explain what a tool is in an agent system.")
    print(result.final_output)

if __name__ == "__main__":
    asyncio.run(main())
```

This is the base unit. The runner executes the agent orchestration, and a single run may still involve multiple internal model/tool steps. (OpenAI GitHub)

Add a tool

```
from agents import Agent, Runner, function_tool

@function_tool
def add(a: int, b: int) -> int:
    """Add two integers."""
    return a + b

agent = Agent(
    name="MathAssistant",
    instructions="Use the add tool when needed.",
    tools=[add],
)

result = await Runner.run(agent, "What is 12 + 30?")
print(result.final_output)
```

Function tools are regular Python functions exposed to the model through the SDK's tool interface. The docs say the SDK uses the function signature and docstring to build the schema. (OpenAI GitHub)

Add a specialist handoff

```
from agents import Agent, Runner

billing_agent = Agent(
    name="BillingAgent",
    instructions="Handle billing and invoice questions."
)

support_agent = Agent(
    name="SupportRouter",
    instructions="Route billing questions to the billing agent.",
    handoffs=[billing_agent],
)

result = await Runner.run(support_agent, "Why was I charged twice?")
print(result.final_output)
print(result.last_agent.name)
```

Handoffs are represented to the model as tools, and the runner executes the delegated agent chain. (OpenAI GitHub)

Add sessions for memory

```
from agents import Agent, Runner, SQLiteSession

agent = Agent(
    name="Tutor",
    instructions="Remember the ongoing discussion."
)

session = SQLiteSession("thread_001")

first = await Runner.run(agent, "My topic is cell biology.", session=session)
second = await Runner.run(agent, "Explain mitosis again, but simpler.",
session=session)

print(second.final_output)
```

Sessions preserve conversation history across runs and remove the need to manually pass prior turns each time. (OpenAI GitHub)

Keep private app state out of model context

```
# Pseudocode structure
app_state = {
    "user_id": 123,
    "auth_scope": "admin",
    "db": db_client,
    "logger": logger,
}
```

OpenAI's docs say application state and dependencies can be passed into a run without being sent to the model, which is the right way to handle authenticated user info, database clients, loggers, and helpers. (OpenAI Developers)

Use manager-style specialists

```
summary_agent = Agent(
    name="Summarizer",
    instructions="Summarize the supplied text briefly and accurately."
)

research_agent = Agent(
    name="Researcher",
    instructions="Extract relevant facts from the supplied text."
)
```

```
main_agent = Agent(
    name="Manager",
    instructions="Use specialists as needed, then produce the final answer.",
    # conceptually: agents as tools
)
```

OpenAI documents `agent.asTool()` / agents-as-tools for manager-style workflows where the main agent remains responsible and specialist agents act as helpers. (OpenAI Developers)

---

5) How to choose the right pattern

Use a single agent when:

- the task is narrow,
- the tools are few,
- the output policy is simple.

Use handoffs when:

- different specializations should own different parts of the conversation,
- the user experience should clearly move between experts.

Use agents-as-tools when:

- one manager must remain in control,
- specialists should only do subwork.

Use sessions when:

- the user expects memory across turns.

Use guardrails when:

- the input or output is risky,
- the tool is powerful,
- the format must be strict.

Use sandbox agents when:

- files, commands, packages, or workspace state are needed.

Use human-in-the-loop when:

- tool actions need approval before execution. (OpenAI GitHub)

---

6) What actually happens under the hood

A run is not just one prompt and one completion. OpenAI says multi-step runs can produce more than one response, for example across handoffs or repeated model/tool/model cycles, and `raw_responses` captures those underlying responses. (OpenAI GitHub)

So the engine may do this:

```
Model decides to call tool
→ tool executes
→ tool result returned to model
→ model decides to hand off
→ specialist agent runs
→ specialist uses another tool
→ final output returned
```

That is normal. That is the framework.

---

7) The key design principle

Do not build one giant prompt. Build a small system:

- **agent** for role definition,
- **tools** for actions,
- **handoffs** for specialization,
- **sessions** for continuity,
- **guardrails** for safety,
- **tracing** for debugging,
- **sandbox** for file/workspace tasks,
- **human approval** for risky actions. (OpenAI GitHub)

That is the architecture.

---

8) The practical takeaway

If you are building with the Agents SDK, this is the clean default layout:

```
Request
  ↓
Guardrail
  ↓
Router / Triage Agent
  ↓
Tools or Handoff or Agent-as-Tool
  ↓
Session updates
  ↓
Tracing
```

```
    ↓
  Final result
```

That is the whole model in one line.